AI-Driven Plant Disease Detection for Smart Agriculture Using CNN

Bachelor of Technology

In

Computer Science and Engineering

By

MD FARIYAZ RAHAMAN - CSE224030

IMTAJ ALI - CSE224033

Under the guidance of Dr. Zeenat Rehena Associate Professor

Department of Computer Science and Engineering,

Aliah University

I-A/27, New Town, Kolkata - 700160

Department of Computer Science and Engineering
Aliah University
I-A/27, New Town, Kolkata - 700160

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled **"AI-Driven Plant Disease Detection for Smart Agriculture Using CNN"** is our original work carried out under the guidance of **Dr. Zeenat Rehena** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech) in Computer Science and Engineering**.

We certify that this work has not been submitted, either wholly or partially, for the award of any degree, diploma, or other academic qualification at any University or Institution.

We further declare that all sources of information, ideas, and materials used in this project have been properly acknowledged and referenced in accordance with standard academic practices.

We affirm that this report is a true record of the work carried out by us and includes all revisions approved by the project guide and review committee.

MD FARIYAZ RAHAMAN – CSE(224030)
IMTAJ ALI – CSE(224033)

Date: 18-06-2026

Department of Computer Science and Engineering
ALIAH UNIVERSITY
I-A/27, New Town, Kolkata - 700160.

CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled "**AI-Driven Plant Disease Detection for Smart Agriculture Using CNN**" submitted by Md. Fariyaz Rahaman (Roll No. CSE224030) and Imtaj Ali (Roll No. CSE224033) of the Department of Computer Science and Engineering is a record of bona fide work carried out by them under the guidance and supervision of Dr. Zeenat Rehena in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering.

The work presented in this project has been examined and found satisfactory. To the best of my knowledge, the project embodies the original work carried out by the students during the academic session and has not been submitted elsewhere for the award of any other degree or diploma.

Signature of Supervisor with Seal:

Dr. Zeenat Rehena

Associate Professor, Dept of CSE

ALIAH UNIVERSITY

Date: 18-06-2026

Department of Computer Science and Engineering
Aliah University
I-A/27, New Town, Kolkata - 700160.

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the Major Project report entitled "**AI-Driven Plant Disease Detection for Smart Agriculture Using CNN**", being submitted by Md. Fariyaz Rahaman (Roll No. CSE224030) and Imtaj Ali (Roll No. CSE224033), in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering, is a record of bona fide work carried out by them under the guidance and supervision of Dr. Zeenat Rehena.

The work presented in this project has been examined and found satisfactory for the fulfillment of the requirements of the degree.

Signature of the Head of the Department:

Dr. SK Md Mosaddek Hossain

Associate Professor & Head of the Department of CSE

ALIAH UNIVERSITY

Date: 18-06-2026

Acknowledgement

We would like to express our sincere gratitude to our project guide, **Dr. Zeenat Rehena**, for her valuable guidance, encouragement, and continuous support throughout the development of this project. Her expertise and suggestions greatly contributed to the successful completion of our work.

We are thankful to the **Head of the Department** and all the faculty members of the **Department of Computer Science and Engineering** for providing the necessary facilities, support, and academic guidance during the course of this project.

We also extend our gratitude to the Project Review Committee members for their constructive suggestions and valuable feedback, which helped us improve the quality of our project.

Finally, we express our heartfelt thanks to our parents, family members, and friends for their constant encouragement, moral support, and motivation throughout our academic journey. Above all, we thank Almighty God for giving us the strength and determination to successfully complete this project.

MD FARIYAZ RAHAMAN - (CSE224030)

IMTAJ ALI - (CSE224033)

DECLARATION

We hereby declare that this project report entitled **"AI-Driven Plant Disease Detection for Smart Agriculture Using CNN"** is the result of our own work carried out under the guidance of **Dr. Zeenat Rehena**.

We confirm that all ideas, data, information, and materials taken from other sources have been properly acknowledged and referenced. We have followed the principles of academic honesty and integrity and have neither copied nor misrepresented any information in this report.

We understand that any violation of academic ethics, including plagiarism or improper citation, may result in disciplinary action by the Institute.

MD FARIYAZ RAHAMAN – (CSE224030)
IMTAJ ALI – (CSE224033)

ABSTRACT

Agriculture plays a vital role in the Indian economy and is the primary source of livelihood for a large portion of the population. Plant diseases significantly affect crop yield and quality, leading to economic losses for farmers. Early and accurate detection of plant diseases is therefore essential for improving agricultural productivity and ensuring food security.

Traditional methods of disease identification rely on manual inspection by experts, which can be time-consuming, costly, and prone to errors. To overcome these limitations, this project proposes a **"AI-Driven Plant Disease Detection for Smart Agriculture Using CNN"** The system utilizes **Convolutional Neural Networks (CNNs)** to identify plant diseases from leaf images with high accuracy.

A web-based application has been developed using **Flask**, allowing users to upload images of plant leaves for disease detection. The trained CNN model analyzes the uploaded image and predicts the disease category. In addition, the system provides information about the disease, including its causes and possible treatment methods through a JSON-based knowledge database.

The model was trained on a plant disease dataset collected from **Kaggle**, containing multiple plant species and disease classes. Experimental results demonstrate that the proposed system achieves high prediction accuracy and provides a simple, fast, and user-friendly solution for plant disease diagnosis. This system can assist farmers, researchers, and agricultural professionals in identifying diseases at an early stage and taking appropriate preventive measures.

Table of Contents

CHAPTER 1
INTRODUCTION

1.1 Introduction:

Agriculture is very important for our economy. It helps feed millions of people and gives them a living.. Plant diseases hurt crop production a lot. They reduce how much we get and the quality of what we grow. Finding plant diseases early is crucial to stop crop losses and make farming more productive.

Usually farmers or experts look at plants to find diseases. This can take a lot of time cost a lot of money and sometimes not be accurate. With AI Machine Learning and Deep Learning getting better we can now use automated systems to detect diseases in plants.[3][4][5]

Our project, "Plant Disease Recognition System Using Deep Learning" aims to identify plant diseases from leaf images. We use a kind of AI called Convolutional Neural Network (CNN). CNN is great at looking at images and figuring out what they are. It can automatically find details in leaf images and tell us what disease the plant has.[9]

The system is built using Python, TensorFlow, Keras, NumPy and Flask. We created a website where users can upload pictures of leaves. The picture is then looked at by our trained CNN model. It predicts what disease the plant has and shows the result away.

To make our system more helpful we added a database that gives information about the disease. This includes what causes it and how to treat it. This helps farmers take the steps to save their crops.

We used a lot of leaf images from Kaggle to train our model. These images are of diseased leaves from many kinds of plants. By looking at many images our model can accurately find different plant diseases.

Our system has benefits. It can detect diseases fast doesn't rely on experts is accurate and easy to use. It supports the idea of farming by helping us find and treat diseases on time.[10]

In short our Plant Disease Recognition System is a reliable and easy-to-use tool. It helps find plant diseases and improves crop health and farming productivity.

The system helps farmers protect their crops. Plant disease detection is crucial, for farmers. The Plant Disease Recognition System uses Deep Learning. Deep Learning helps in image classification. Image classification is used to identify plant disease.[2][6][9]

1.2 Background and Motivation:

Agriculture is one of the most important sectors of the economy and plays a vital role in ensuring food security and supporting livelihoods. However, plant diseases remain a major challenge for farmers,

causing significant reductions in crop yield and quality. Diseases caused by fungi, bacteria, viruses, and pests can spread rapidly if not detected at an early stage, leading to severe economic losses.

Traditionally, plant disease identification relies on visual inspection by farmers or agricultural experts. This method requires experience and knowledge and may not always provide accurate results. In many rural areas, access to agricultural specialists is limited, making timely disease diagnosis difficult. As a result, farmers may apply incorrect treatments, leading to increased costs and reduced productivity.

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning have created new opportunities for developing automated disease detection systems. Image processing and computer vision techniques can analyze plant leaf images and identify diseases with high accuracy.

Among these techniques, Convolutional Neural Networks (CNNs) have shown excellent performance in image classification tasks due to their ability to automatically extract important features from images.

The motivation behind this project is to develop an intelligent and user-friendly Plant Disease Recognition System that can assist farmers and agricultural practitioners in identifying plant diseases quickly and accurately. By using a CNN-based deep learning model and a Flask web application, the system allows users to upload leaf images and receive instant disease predictions along with treatment information.

This project aims to reduce dependence on manual diagnosis, minimize crop losses, support timely disease management, and contribute to the adoption of modern technologies in agriculture. The system also promotes precision agriculture by enabling efficient monitoring of crop health and improving overall agricultural productivity.[4][10]

1.3 Scope of the Research:

The scope of this research is focused on the development of an automated Plant Disease Recognition System using Deep Learning techniques. The system is designed to identify diseases in plant leaves by analyzing digital images and providing accurate predictions through a trained Convolutional Neural Network (CNN) model.

This research covers the collection and utilization of plant leaf image datasets obtained from Kaggle for training and testing purposes. The study includes image preprocessing, model training, disease classification, and the development of a user-friendly web application using the Flask framework.

The system enables users to upload plant leaf images and receive instant disease detection results.

In addition to disease identification, the research incorporates a JSON-based database to provide information about the detected disease, including its causes and recommended treatment methods.

This feature makes the system more practical and beneficial for farmers and agricultural users.

The project is applicable to various plant species and diseases included in the dataset. It aims to support precision agriculture by enabling faster disease diagnosis, reducing dependency on agricultural experts, and helping farmers take timely preventive measures.

Although the system achieves high accuracy in disease detection, its performance depends on the quality and diversity of the training dataset. The current research is limited to diseases present in the dataset and image-based diagnosis only. Future enhancements can include real-time disease detection through mobile devices, integration with IoT sensors, and support for a larger number of plant species and diseases.

1.4 Advantages of the Research:

The proposed Plant Disease Recognition System offers several advantages in the field of agriculture by providing an automated and intelligent approach for disease detection. The major advantages of this research are as follows:

1. Early Disease Detection: The system can identify plant diseases at an early stage, helping farmers take preventive measures before the disease spreads widely.
2. High Accuracy: By using a Convolutional Neural Network (CNN), the system can classify plant diseases with high accuracy and reliability.
3. Time Saving: Automated disease detection reduces the time required for manual inspection and diagnosis by agricultural experts.
4. Cost Effective: Farmers can obtain disease information without frequently consulting specialists, reducing diagnosis and treatment costs.
5. User-Friendly Interface: The Flask-based web application provides a simple interface where users can easily upload leaf images and receive results.
6. Reduced Human Error: The system minimizes errors that may occur during manual disease identification.
7. Instant Results: Disease prediction and information are generated quickly, enabling faster decision-making.
8. Treatment Recommendations: Along with disease detection, the system provides information about causes and possible remedies using a JSON database.
9. Supports Precision Agriculture: The research promotes the use of Artificial Intelligence in agriculture for efficient crop monitoring and management.
10. Improved Crop Productivity: Timely disease detection and treatment help reduce crop losses and improve overall agricultural yield.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview:

Plant diseases significantly affect agricultural productivity and crop quality. Early detection of diseases is essential for preventing crop losses and improving yield. Researchers have proposed various techniques for plant disease detection, ranging from traditional image processing methods to advanced deep learning approaches. This chapter reviews important research works related to plant disease detection and classification using image processing, machine learning, and deep learning techniques.

2.2 Traditional Plant Disease Detection Approaches:

Early research in plant disease detection mainly focused on image processing techniques such as image segmentation, color analysis, texture analysis, and feature extraction.

Sachin D. Khirade et al.[11] presented a survey on plant disease detection using image processing techniques. The study highlighted the importance of disease identification in agriculture and discussed various stages including image acquisition, preprocessing, segmentation, feature extraction, and classification. The authors concluded that Artificial Neural Networks (ANN), Support Vector Machines (SVM), and other machine learning algorithms can effectively classify plant diseases from leaf images.

Amandeep Singh [12] focused on practical challenges associated with image acquisition. The study emphasized the importance of obtaining high-quality leaf images and handling illumination variations. The research demonstrated that image quality plays a critical role in achieving accurate disease detection.

2.3 Machine Learning Based Approaches:

Several researchers have applied machine learning algorithms for plant disease classification.

Satish Madhgoria [13] proposed an automatic pixel-based classification method for detecting unhealthy regions in leaf images. The method used image segmentation followed by Support

improve classification accuracy by eliminating falsely classified pixels. The study demonstrated that SVM-based approaches can effectively identify diseased regions based on color information.

Although machine learning techniques provide satisfactory performance, their effectiveness depends heavily on handcrafted feature extraction methods. Selecting appropriate features often requires domain expertise and may limit the model's ability to generalize to unseen data.

2.4 Deep Learning and CNN-Based Approaches:

Recent advances in Deep Learning have significantly improved the performance of plant disease recognition systems. Convolutional Neural Networks (CNNs) automatically learn features from images without requiring manual feature engineering.

Robert G. de Luna et al.[5] developed an automated image capturing system for tomato leaf disease detection. The system utilized CNN-based models for identifying diseases such as Phoma Rot, Leaf Miner, and Target Spot. Experimental results showed a disease recognition accuracy of 95.75%, demonstrating the effectiveness of deep learning methods in agricultural applications.

Omkar Kulkarni [14] proposed a deep learning-based crop disease detection system trained on publicly available datasets. The model classified healthy and diseased crop leaves based on visual patterns and texture characteristics. The study highlighted the capability of CNN models to achieve high classification accuracy while reducing human intervention.

Karunya Rathan et al. [15] developed an image processing and deep learning-based disease detection system involving image acquisition, preprocessing, segmentation, feature extraction, and classification. The research demonstrated that automated systems can significantly assist farmers in identifying plant diseases quickly and accurately.

2.5 Comparative Analysis:

Traditional image processing methods provide useful results but often depend on manual feature extraction and controlled environmental conditions. Machine learning approaches such as SVM improve classification performance but still require careful feature engineering. Deep learning models, particularly Convolutional Neural Networks, automatically learn relevant features from images and generally outperform traditional approaches in terms of accuracy,

scalability, and robustness.

CNN-based systems have shown superior performance in handling large datasets and complex disease patterns. These models can accurately classify multiple disease categories while minimizing human effort and reducing the chances of misclassification.

2.6 Research Gap:

Most existing studies focus primarily on disease classification and do not provide additional information about disease causes and treatment recommendations. Some systems also require specialized hardware or are limited to specific crops and diseases.

To address these limitations, the proposed Plant Disease Recognition System integrates a CNN-based classification model with a Flask web application and a JSON-based disease information database. The system not only predicts diseases but also provides information regarding causes and remedies, making it more practical and useful for farmers and agricultural users.

2.7 Summary:

The literature survey indicates that deep learning techniques, especially Convolutional Neural Networks, offer significant advantages over traditional image processing and machine learning methods for plant disease detection. Based on the findings of previous studies, a CNN-based Plant Disease Recognition System has been developed to provide accurate disease classification along with disease information and treatment recommendations through a user-friendly web interface.

Chapter 3
Convolutional Neural Network

3.1 NEURAL NETWORKS:

Artificial Neural Networks (ANNs) are computing systems inspired by the structure and functioning of the human brain. They consist of interconnected processing units called neurons that work together to solve complex problems such as classification, prediction, and pattern recognition.

In an ANN, information flows from the input layer through one or more hidden layers and finally reaches the output layer. Each neuron receives input values, performs mathematical computations, and passes the result to the next layer. During training, the network adjusts its internal weights to improve prediction accuracy.

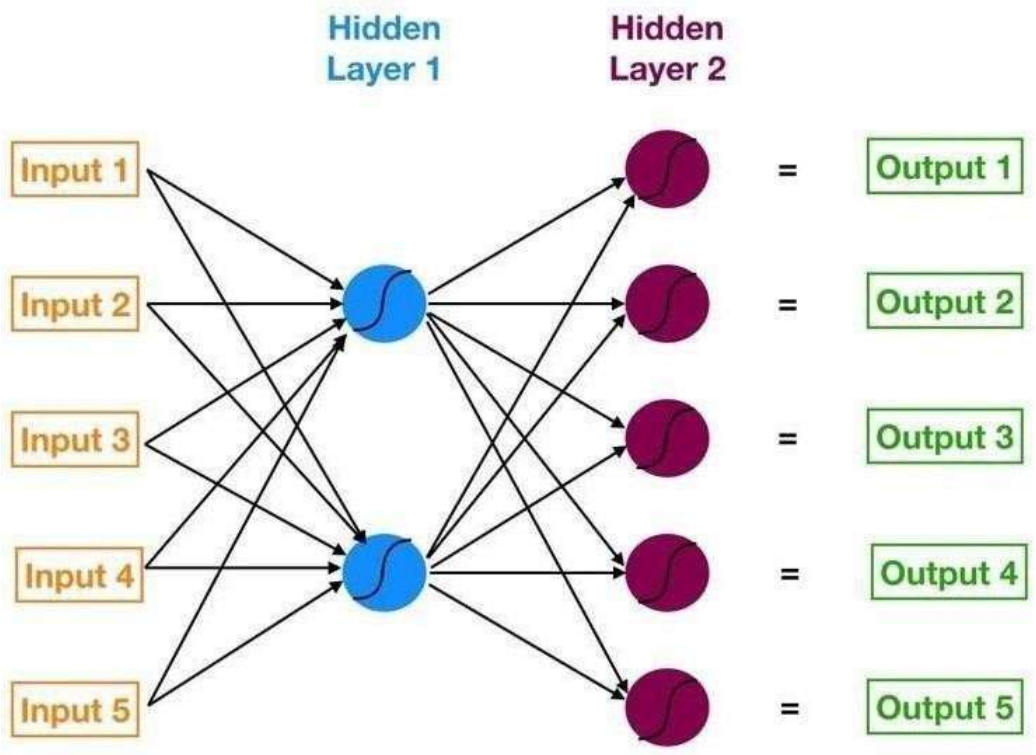

Figure 3.1: The Layers in the Neural Network

ANNs are widely used in Machine Learning applications because they can learn complex relationships from large datasets and make accurate predictions. In this project, ANN concepts form the foundation of the Convolutional Neural Network (CNN) model used for plant disease classification.[6][7]

3.2 NEURONS:

A neuron is the basic processing unit of an artificial neural network. Each neuron receives one or more inputs, multiplies them by corresponding weights, adds a bias value, and then passes the result through an activation function.

The operation of a neuron can be expressed as:

Output = Activation Function (Weighted Sum of Inputs + Bias)

The weights determine the importance of each input, while the bias helps shift the output value. By adjusting these parameters during training, the neural network learns to recognize patterns in data.

Neurons work collectively across multiple layers to analyze images and classify plant diseases accurately.

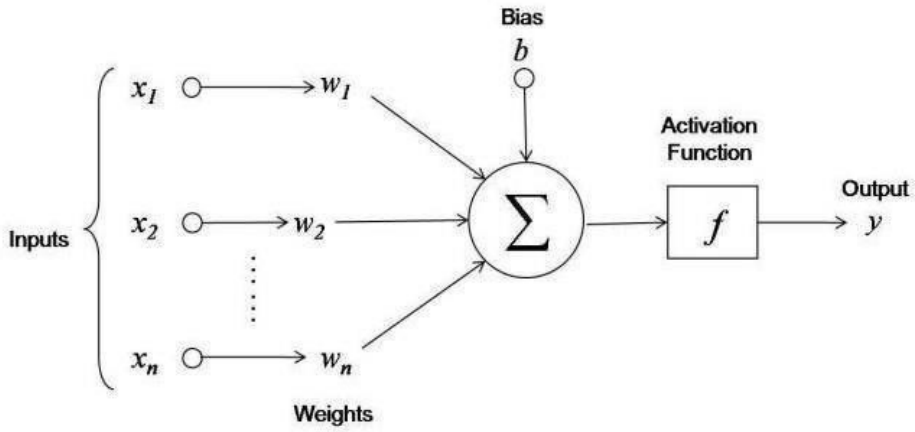

Figure 3.2: The internal structure of neural network with bias and activation function

The functioning of a neuron can be summarized in the following steps:

- Receive input values from the previous layer.
- Multiply each input by its corresponding weight.
- Add all weighted values together.
- Add a bias value to the result.
- Apply an activation function.
- Pass the output to the next layer.

The learning capability of neural networks depends largely on the proper adjustment of neuron weights and biases. Through repeated training, neurons gradually learn important features present in plant leaf images and contribute to accurate disease prediction.[6]

3.3 ACTIVATION FUNCTIONS: Activation functions introduce non-linearity into the neural network, allowing it to learn complex patterns from data. Without activation functions, a neural network would behave like a simple linear model.

Activation functions determine whether a neuron should be activated and what output should be forwarded to the next layer. They play a crucial role in improving the learning capability of deep learning models.

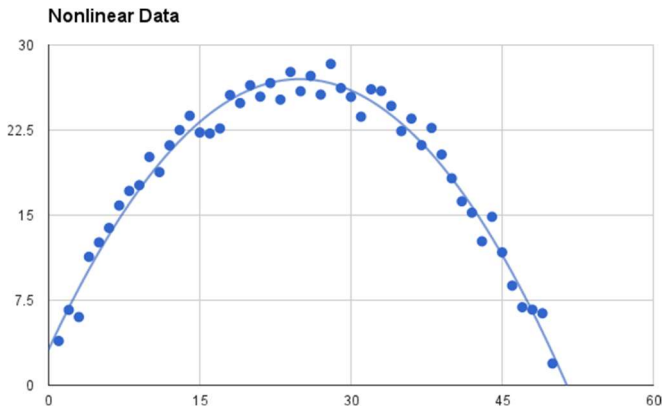

Figure 3.3 Activation Functions

- **Rectified Linear Unit (ReLU) Activation Function :** The Rectified Linear Unit (ReLU) is one of the most commonly used activation functions in Deep Learning. It is mathematically defined as:
- ReLU(x) = max(0, x)
- If the input value is positive, the output remains unchanged. If the input value is negative, the output becomes zero.

Advantages of ReLU

- Computationally efficient
- Reduces training time
- Helps solve the vanishing gradient problem
- Improves model performance in deep networks
- Enables faster convergence during training

Because of its simplicity and effectiveness, ReLU has become the standard activation function for most CNN architectures.

In this project, ReLU activation is used in the convolutional layers of the CNN model.

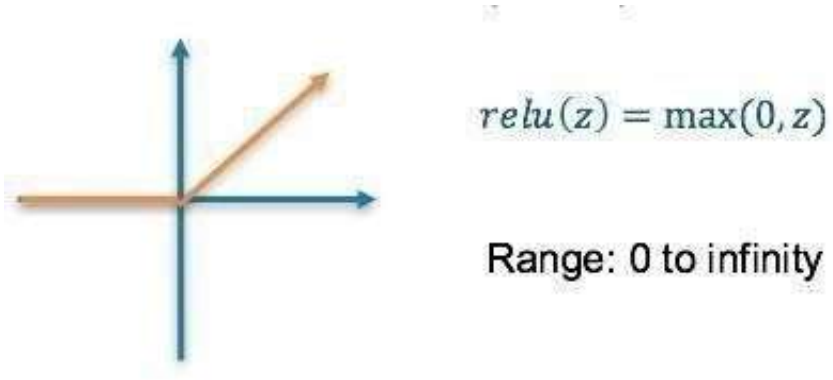

Figure 3.4 ReLU

Softmax Activation Function : The Softmax activation function is used in the output layer of multi-class classification models. It converts output values into probability scores ranging from 0 to 1.

The class with the highest probability is selected as the final prediction.

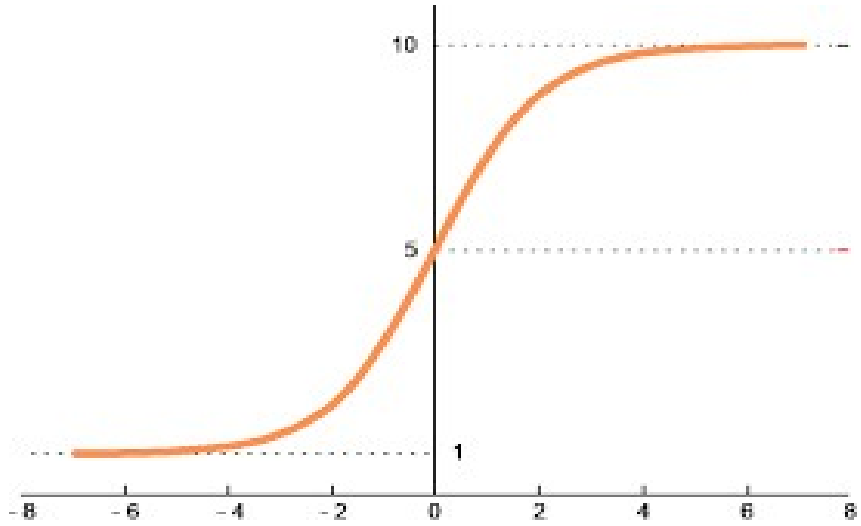

Figure 3.5 Softmax Activation

Advantages of Softmax

- Produces probability-based outputs
- Suitable for multi-class classification
- Helps determine prediction confidence
- Improves interpretability of model outputs

Softmax Activation Function : The Softmax activation function is used in the output layer of multi-class classification models. It converts output values into probability scores ranging from 0 to 1.

The class with the highest probability is selected as the final prediction.

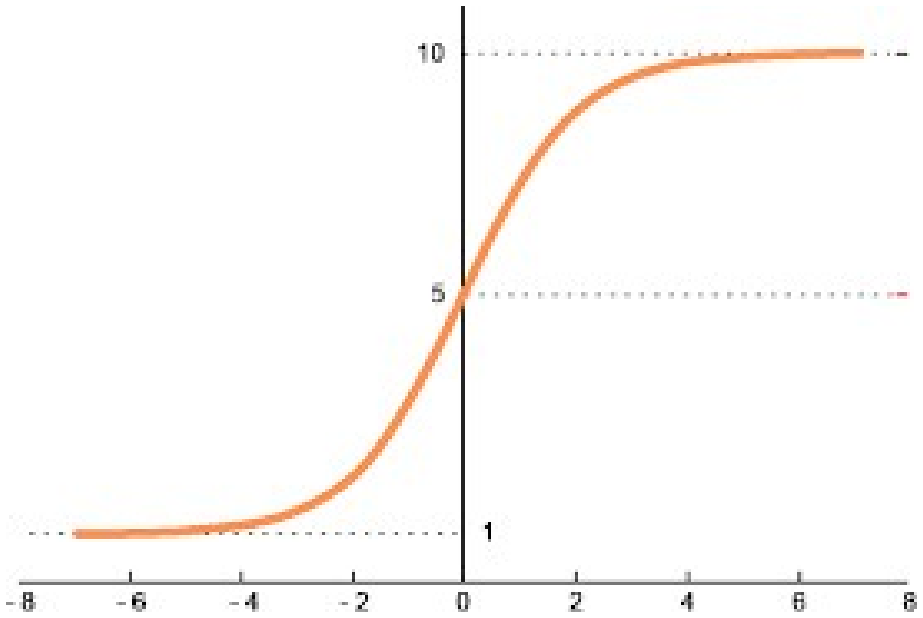

Figure 3.6: Sigmoid Activation Function Graph

Advantages of Softmax

- Produces probability-based outputs
- Suitable for multi-class classification
- Helps determine prediction confidence
- Improves interpretability of model outputs

In the Plant Disease Recognition System, Softmax is used to classify leaf images into multiple disease categories and healthy classes.[6]

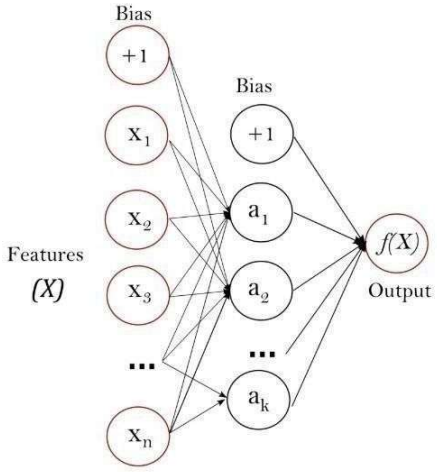

Figure 3.7: Architecture of an Artificial Neural Network (ANN)

3.4 CONVOLUTIONAL NEURAL NETWORK (CNN):

Convolutional Neural Network (CNN) is a specialized type of Deep Learning model designed specifically for image processing and classification tasks. CNN automatically extracts important features from images, such as edges, textures, colors, and patterns, making it highly effective for plant disease detection.

Unlike traditional machine learning techniques, CNN does not require manual feature extraction. The model automatically learns relevant features directly from the training images. This capability makes CNN highly suitable for agricultural applications where disease symptoms may vary significantly.[2][9]

The CNN architecture used in this project consists of several layers:

Convolution Layer

The convolution layer extracts important visual features from the input image using filters. These filters detect disease-related patterns such as spots, discoloration, texture variations, and damaged regions on plant leaves.

Activation Layer

After feature extraction, the ReLU activation function is applied to introduce non-linearity into the model. This enables the network to learn more complex image patterns.

Pooling Layer

Pooling reduces the size of feature maps while preserving important information. This helps reduc

computational complexity, memory usage, and training time while improving model efficiency.

Fully Connected Layer

The fully connected layer combines the extracted features and performs classification based on the learned patterns. It acts as the decision-making component of the network.

Output Layer

The output layer uses the Softmax activation function to predict the probability of each disease class and identify the most likely disease.

3.5 ROLE OF CNN IN THE PROJECT

In the proposed Plant Disease Recognition System, CNN serves as the core classification model. The network is trained using thousands of plant leaf images collected from the Kaggle dataset. During training, the CNN learns disease-specific features automatically and develops the ability to distinguish between healthy and diseased leaves.

The trained model can identify symptoms such as leaf spots, color changes, lesions, and texture abnormalities that are commonly associated with plant diseases. This enables the system to perform disease detection with high accuracy and reliability.

When a user uploads a leaf image through the Flask web application, the trained CNN model analyzes the image and predicts the disease category. The prediction result is then displayed along with disease information, causes, and treatment recommendations.

The use of CNN significantly improves disease detection accuracy and provides an efficient solution for automated plant disease diagnosis in agriculture. By combining Deep Learning, Computer Vision, and Web Technologies, the proposed system helps farmers and agricultural professionals detect diseases early, reduce crop losses, and improve overall agricultural productivity.[2][9][10].

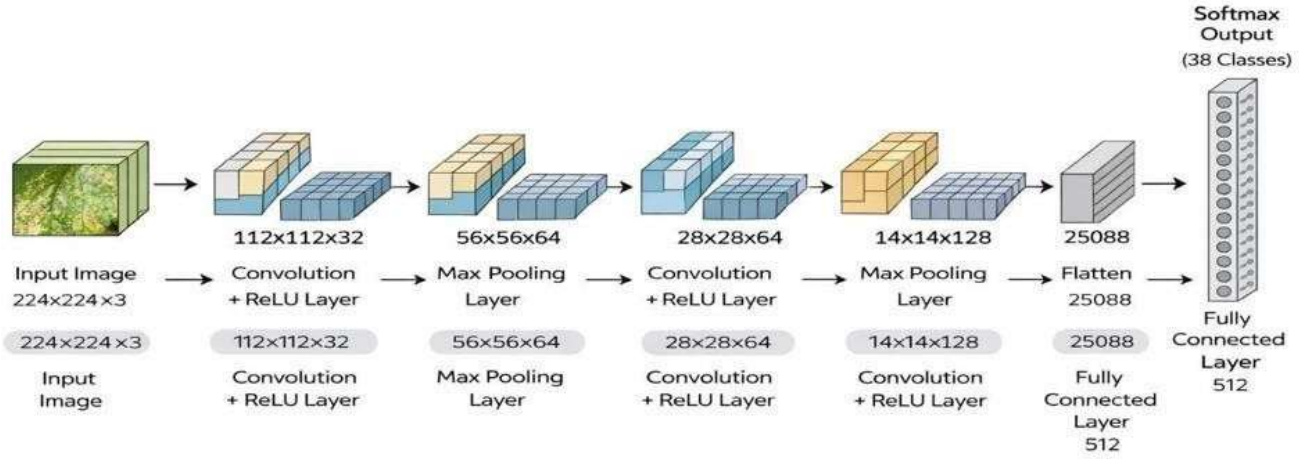

Figure 3.8 : CNN Model Architecture

Chapter 4

Plant Disease Detection using CNN

Agriculture plays a crucial role in the economy of many countries, especially India, where a large portion of the population depends on farming for their livelihood. The quality and quantity of agricultural production are significantly affected by various factors such as climate change, soil conditions, pests, and plant diseases. Among these factors, plant diseases are one of the major causes of crop loss, resulting in reduced productivity and economic losses for farmers.

Early and accurate detection of plant diseases is essential for effective crop management. Traditional disease detection methods rely on manual observation by farmers or agricultural experts. These methods are often time-consuming, expensive, and may lead to incorrect diagnosis due to a lack of expertise or delayed identification. Improper disease recognition can result in the excessive use of pesticides and fertilizers, which may increase production costs and negatively impact the environment.

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), and Computer Vision have opened new possibilities for automated plant disease detection. Deep Learning techniques, particularly Convolutional Neural Networks (CNNs), have demonstrated excellent performance in image classification and pattern recognition tasks. These technologies enable computers to learn complex features directly from images and accurately identify different plant diseases.

The widespread availability of smartphones, digital cameras, and internet connectivity has further increased the feasibility of image-based disease diagnosis systems. Farmers can capture images of infected plant leaves and use intelligent software applications to obtain instant disease predictions and treatment recommendations. Such systems help reduce dependence on agricultural experts and provide faster decision-making support.

The proposed project, **" AI-Driven Plant Disease Detection for Smart Agriculture Using CNN"**, utilizes a CNN-based model to identify plant diseases from leaf images. A web application developed using Flask provides an easy-to-use interface where users can upload plant leaf images for analysis. The system predicts the disease category and displays relevant information such as disease causes and treatment methods.

The dataset used for training the model was obtained from Kaggle and contains images of healthy and diseased leaves from various plant species. Through image preprocessing, model training, and prediction techniques, the system achieves high accuracy in disease classification. The proposed solution offers a reliable, efficient, and user-friendly approach for plant disease diagnosis and can contribute significantly to modern agricultural practices and precision farming.

4.1 Proposed Work

The proposed Plant Disease Recognition System is designed to automatically identify diseases affecting plant leaves using Deep Learning techniques. Agriculture is highly dependent on the health of crops, and plant diseases can significantly reduce both the quality and quantity of agricultural production. Traditional methods of disease identification rely on visual inspection by farmers or agricultural experts, which may be time-consuming, expensive, and prone to human error. To overcome these challenges, the proposed system utilizes image processing and Convolutional Neural Networks (CNNs) to provide a fast, accurate, and automated solution for plant disease detection.

The system is capable of recognizing diseases from images of plant leaves and providing detailed information regarding the identified disease. The application has been developed using Flask, which provides a simple web-based interface for uploading leaf images and obtaining prediction results. The disease information, including causes and treatment recommendations, is stored in a JSON database and displayed to the user after successful prediction.

The complete methodology of the proposed system consists of several stages, including dataset collection, image preprocessing, model development, training, prediction, disease information retrieval, and web application integration.[2][9]

4.1 Framework:

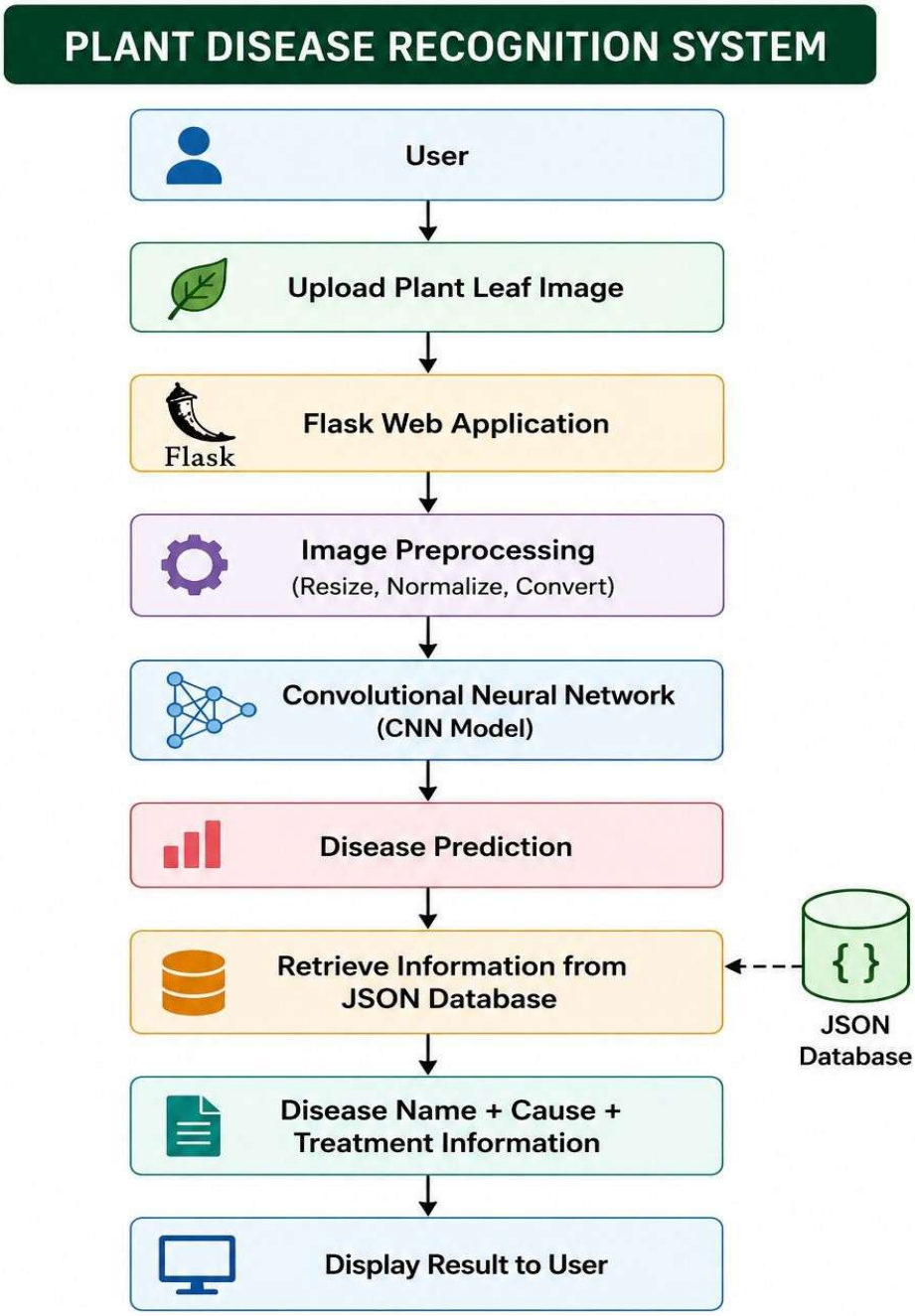

Figure 4.1 : Workflow of the Plant Disease Recognition System

4.2 Dataset Collection

The first stage of the project involves collecting a suitable dataset for training the deep learning model. The dataset used in this project was obtained from Kaggle and contains thousands of images of healthy and diseased plant leaves. The dataset includes multiple plant species such as Apple, Corn, Grape, Potato, Tomato, Strawberry, Orange, Peach, and Pepper. Each image is categorized according to a specific disease class or healthy condition.

A diverse dataset is essential because it enables the model to learn different disease characteristics and improves its ability to generalize to unseen data. The dataset contains images captured under different conditions, allowing the model to recognize disease patterns more effectively [9].

Figure 4.2: Dataset Sample Image

4.3 Data Preprocessing

Before training the model, the collected images undergo preprocessing. Raw images often contain variations in size, brightness, and quality, which can negatively affect model performance. Therefore, all images are resized to a fixed dimension suitable for CNN processing.

The pixel values of images are normalized to improve computational efficiency and ensure stable model training. Image preprocessing helps reduce noise and standardize the dataset, making it easier for the model to learn important disease-related features. These preprocessing operations improve the overall accuracy and reliability of the system.[9]

Convolutional Neural Network (CNN)

The core component of the proposed system is the Convolutional Neural Network (CNN). CNN is a specialized Deep Learning architecture designed for image analysis and classification tasks. Unlike traditional machine learning algorithms, CNN automatically extracts important features from images without requiring manual feature engineering.

The CNN model consists of several layers, including:

- Convolution Layers
- Activation Layers (ReLU)

- Pooling Layers
- Fully Connected Layers
- Output Layer (Softmax)

The convolution layers identify important visual patterns such as disease spots, discoloration, texture changes, and leaf deformations. Pooling layers reduce dimensionality while preserving essential information. Fully connected layers perform classification based on the extracted features. Finally, the Softmax output layer predicts the probability of each disease class and selects the most likely disease category.[6][7]

Model Training and Evaluation

After preprocessing, the dataset is divided into training and testing datasets. The training dataset is used to teach the CNN model how to recognize disease patterns, while the testing dataset is used to evaluate the model's performance.

During training, the CNN adjusts its internal parameters through multiple iterations known as epochs. The model continuously learns from errors and improves its prediction capability. Performance metrics such as training accuracy and validation accuracy are monitored throughout the training process.

The trained model achieved high accuracy in identifying plant diseases, demonstrating the effectiveness of Deep Learning for agricultural image classification tasks. Once the desired performance is achieved, the trained model is saved for deployment within the web application.[6]

Disease Prediction Process

When a user uploads a leaf image through the application, the image undergoes the same preprocessing steps used during training. The processed image is then provided as input to the trained CNN model.

The model analyzes the image and compares its features with patterns learned during training. Based on this analysis, the model predicts the disease category with the highest probability score. The prediction result is then passed to the application for display.

This automated prediction process requires only a few seconds and eliminates the need for manual disease diagnosis.[2][6]

Disease Information Retrieval

Identifying the disease alone may not be sufficient for users. Therefore, the system includes an additional disease information module. A JSON file is used as a lightweight database containing information about different plant diseases.

Once the disease is predicted, the corresponding details are retrieved from the JSON file. The system displays:

- Disease Name
- Disease Cause
- Symptoms
- Recommended Treatment Methods

This feature helps users understand the disease and take appropriate preventive or corrective actions to protect their crops.[10]

Flask Web Application Integration

To make the system accessible and user-friendly, the trained CNN model is integrated with a Flask web application. Flask acts as the backend framework responsible for handling image uploads, prediction requests, and result generation.

The web application consists of several components:

- Home Page
- Image Upload Interface
- Prediction Module
- Result Display Page

Users simply upload a plant leaf image through the interface, and the system automatically performs disease detection and displays the results. The web-based approach ensures ease of use and does not require users to have technical knowledge of machine learning or deep learning.[2][10]

Result Generation

The final stage of the system involves displaying the prediction results. After successful classification, the system presents the disease name along with its confidence score. Additional disease-related information obtained from the JSON database is also displayed.

The generated output provides users with a comprehensive understanding of the detected disease and assists them in making informed decisions regarding disease management and crop protection.

Overall, the proposed methodology combines Deep Learning, Image Processing, and Web Technologies to create an intelligent, efficient, and practical Plant Disease Recognition System. The integration of CNN-based disease classification with a Flask web application provides a reliable solution for early disease detection and supports the adoption of modern technologies in agriculture.

4.4 PACKAGES :

- **NumPy:**

NumPy, which stands for Numerical Python, is a fundamental library used for numerical and array-based computations in Python. It provides support for multidimensional arrays and mathematical operations on these arrays. In this project, NumPy is used to convert plant leaf images into numerical arrays so that they can be processed and provided as input to the CNN model. It also helps in performing normalization and other image preprocessing operationsefficiently.

- **Keras:**

Keras is a high-level deep learning library that runs on top of TensorFlow. It provides a simple and user-friendly interface for designing neural network architectures. In this project, Keras is used to build the CNN model by defining convolutional layers, pooling layers, activation functions, and fully connected layers. It simplifies the process of model creation, training, and evaluation.

- **Matplotlib:**

Matplotlib is a popular Python library used for data visualization and plotting. It helps in generating graphs, charts, and visual representations of data. In this project, Matplotlib is used to visualize training and validation accuracy, loss curves, and sample plant leaf images from the dataset. These visualizations help in analyzing the performance of the CNN model during training.

- **Flask:**

Flask is a lightweight Python web framework used for developing web applications. In this project, Flask is used to create the web-based user interface for the Plant Disease Recognition System. It handles image uploads, processes user requests, communicates with the trained CNN model, and displays prediction results along with disease information in a user-friendly manner.

- **JSON:**

JSON is a lightweight data format used for storing and exchanging structured information. In this project, a JSON file acts as a disease information database. It stores details such as disease names, causes, symptoms, and treatment methods. After the CNN model predicts a disease, the corresponding information is retrieved from the JSON file and displayed to the user through the

Flask web application. JSON provides a simple and efficient way to manage disease-related data without using a complex database system. It also allows easy modification and updating of disease records whenever new information needs to be added to the system.

- **Tensorflow:**

TensorFlow is an open-source deep learning framework developed by Google. It is widely used for building, training, and deploying neural network models. In this project, TensorFlow is used as the backend framework for implementing the Convolutional Neural Network (CNN) model for plant disease classification. It provides efficient computation on CPUs and GPUs and supports the training of deep learning models with high performance.

- **OpenCV / TensorFlow Image Utilities:**

For image preprocessing, TensorFlow image utilities are primarily used in this project to load, resize, and convert images into arrays suitable for model prediction. These utilities help maintain consistency between training and prediction phases and ensure that uploaded images are processed correctly before classification.

- **Werkzeug:**

Werkzeug is a Python library used by Flask for handling web requests and secure file uploads. In this project, it is used to safely upload and store plant leaf images submitted by users through the web application before they are processed by the CNN model for disease prediction.

4.5 PLATFORMS:

- **Google colab:**

Google Colab is a cloud-based platform developed by Google Research for writing and executing Python code through a web browser. It provides a Jupyter Notebook environment without requiring any software installation and offers free access to computational resources such as CPUs, GPUs, and TPUs.

In this project, Google Colab was used for training the Convolutional Neural Network (CNN) model on the plant disease dataset obtained from Kaggle. The availability of GPU support helped in reducing the training time and improving the efficiency of model development. Colab also made it easy to store datasets on Google Drive, execute deep learning code, and monitor training performance from any device with an internet connection.

- **Jupyter notebook :**

Jupyter Notebook is an open-source interactive development environment that allows users to create and execute Python code while documenting the entire workflow in a single file. It combines code, text, mathematical equations, visualizations, and outputs in an organized manner.

In this project, Jupyter Notebook was used for data preprocessing, image analysis, model testing, and performance evaluation. It provided an interactive environment for experimenting with different CNN configurations and visualizing training results such as accuracy and loss graphs. The notebook interface made the development process easier by allowing code execution and result analysis in a step-by-step manner.

Both Google Colab and Jupyter Notebook played an important role in the successful development and testing of the Plant Disease Recognition System.

- **Kaggle :**

Kaggle is an online platform that provides datasets, notebooks, and machine learning resources for data science projects. In this project, the Plant Disease Dataset was collected from Kaggle. The dataset contains images of healthy and diseased plant leaves belonging to different categories. These images were used for training and testing the CNN model. Kaggle provided a reliable and well-organized dataset that helped improve the accuracy and performance of the disease recognition system.

- **Visual Studio Code (VS Code) :**

Visual Studio Code (VS Code) is a lightweight and powerful source-code editor developed by Microsoft. It provides features such as syntax highlighting, debugging, code completion, and extension support for various programming languages. In this project, VS Code was used for developing the Flask web application, integrating the trained CNN model, managing project files, and testing the complete Plant Disease Recognition System. Its user-friendly interface and debugging tools helped in efficient project development and deployment.

Platform Summary

1. **Google Colab** – Training the CNN model with GPU support.
2. **Jupyter Notebook** – Data preprocessing and model experimentation.
3. **Kaggle** – Source of the Plant Disease Dataset.
4. **Visual Studio Code (VS Code)** – Development of the Flask web application and system integration.

4.6 FLOW CHART:

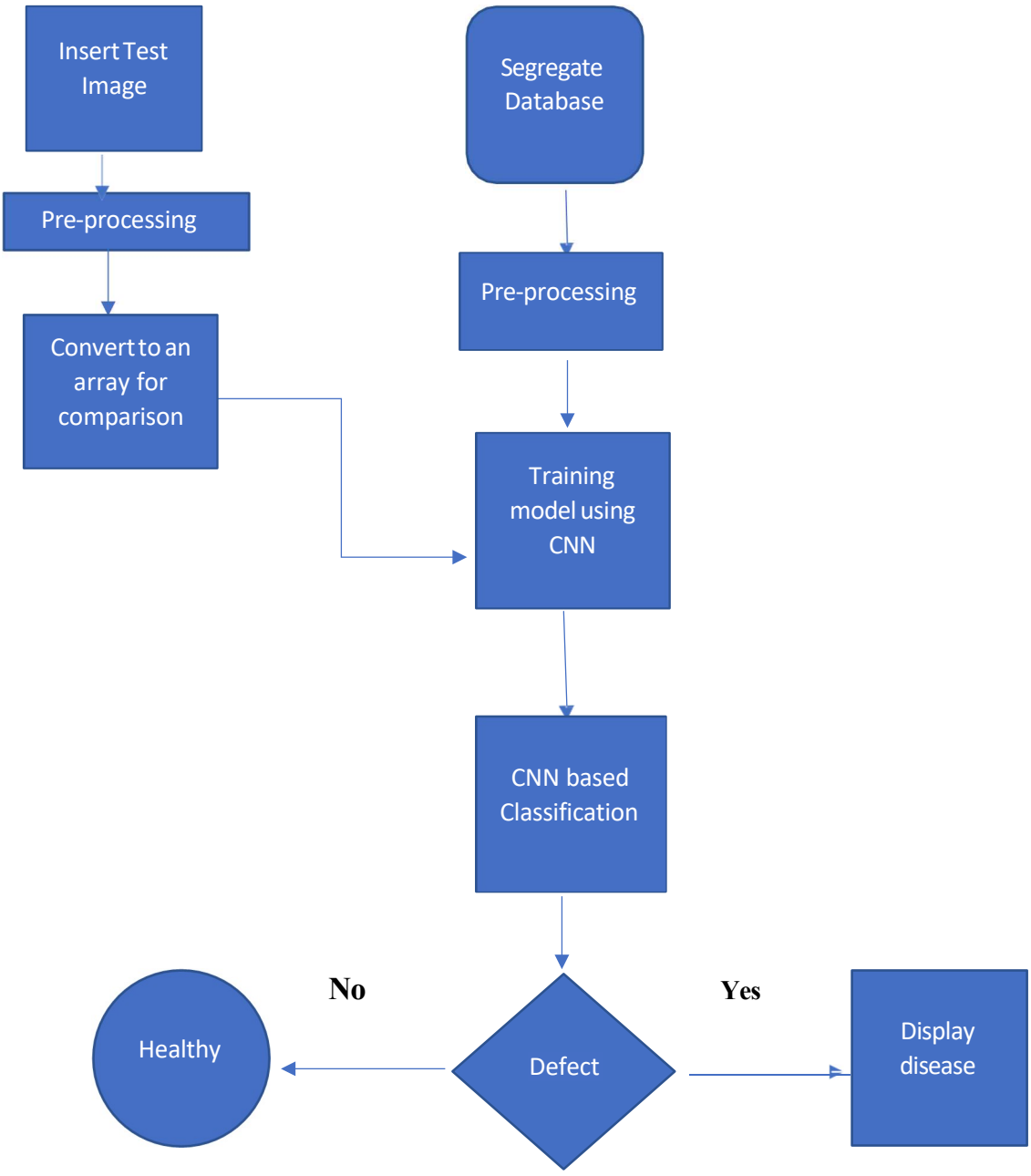

Figure 4.3 flowchart of plant disease recommendation system

CHAPTER 5
Simulation Results and Analysis

5.1 Result & discussion :

The Plant Disease Recognition System was successfully designed and implemented using Deep Learning techniques. A Convolutional Neural Network (CNN) model was trained on a plant leaf image dataset collected from Kaggle to identify various plant diseases accurately. The trained model was integrated with a Flask-based web application to provide an easy-to-use interface for users.
The system accepts a plant leaf image as input and performs image preprocessing before passing it to the trained CNN model. The model analyzes the visual characteristics of the leaf, extracts relevant features, and predicts the corresponding disease category. After prediction, the application retrieves disease-related information from the JSON database and displays the disease name, cause, and recommended treatment measures.[2][9]

The developed system was tested using different plant leaf images belonging to various disease categories. The prediction results obtained were accurate and consistent with the expected outputs. The model successfully identified diseased leaves as well as healthy leaves and generated results within a short processing time. The integration of deep learning with the web application enabled real-time disease detection and improved user accessibility.

The experimental results demonstrate that the proposed system can effectively assist in the early identification of plant diseases. Early disease detection helps farmers and agricultural practitioners take preventive measures at the right time, reducing crop loss and improving productivity. The web-based interface further enhances usability by allowing users to upload images and receive disease information without requiring any technical knowledge.[2][10]

Overall, the Plant Disease Recognition System achieved satisfactory performance and proved to be an efficient, reliable, and user-friendly solution for automated plant disease diagnosis. The results confirm that Deep Learning-based approaches can significantly improve the accuracy and speed of plant disease detection compared to traditional manual inspection methods.[2][9][10]

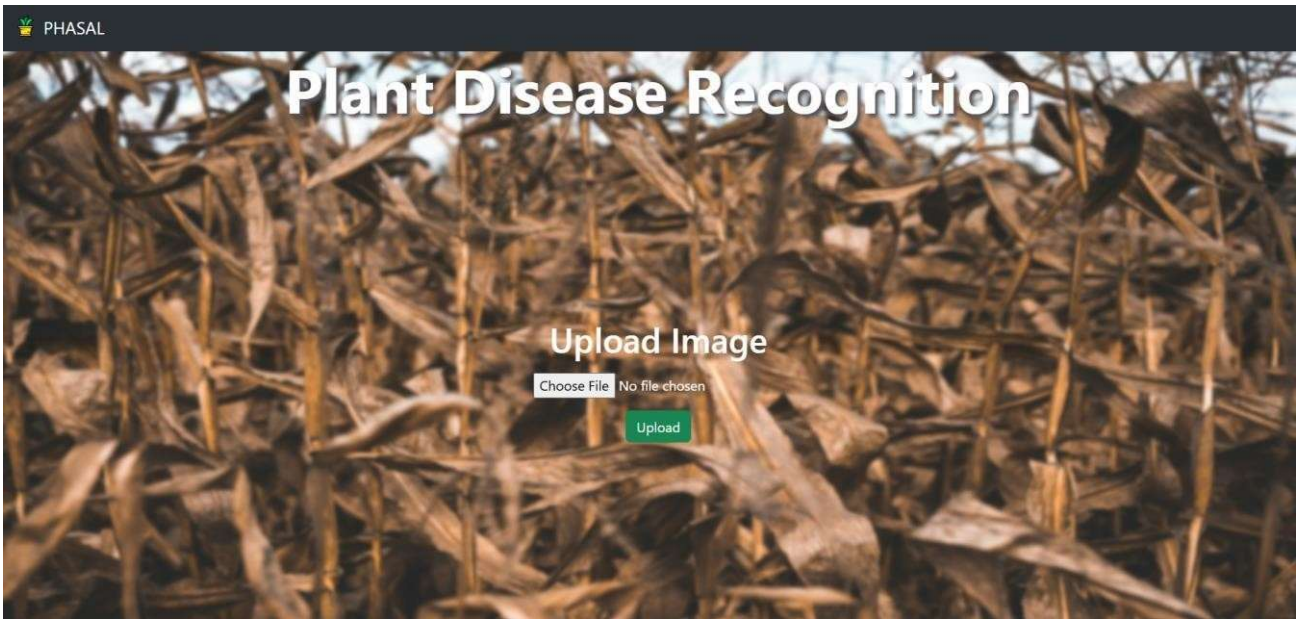

Figure 5.1.1 UI Interface

Home Page of Plant Disease Recognition System

The above figure shows the home page of the Plant Disease Recognition System developed using Flask and Deep Learning. The interface provides a simple and user-friendly platform where users can upload images of plant leaves for disease detection. The page contains an image upload option and an upload button that allows users to submit leaf images to the trained CNN model. After uploading an image, the system processes the input and predicts the corresponding plant disease. The web interface is designed to make disease diagnosis easy and accessible for farmers, researchers, and agricultural practitioners.

Test Image 1)

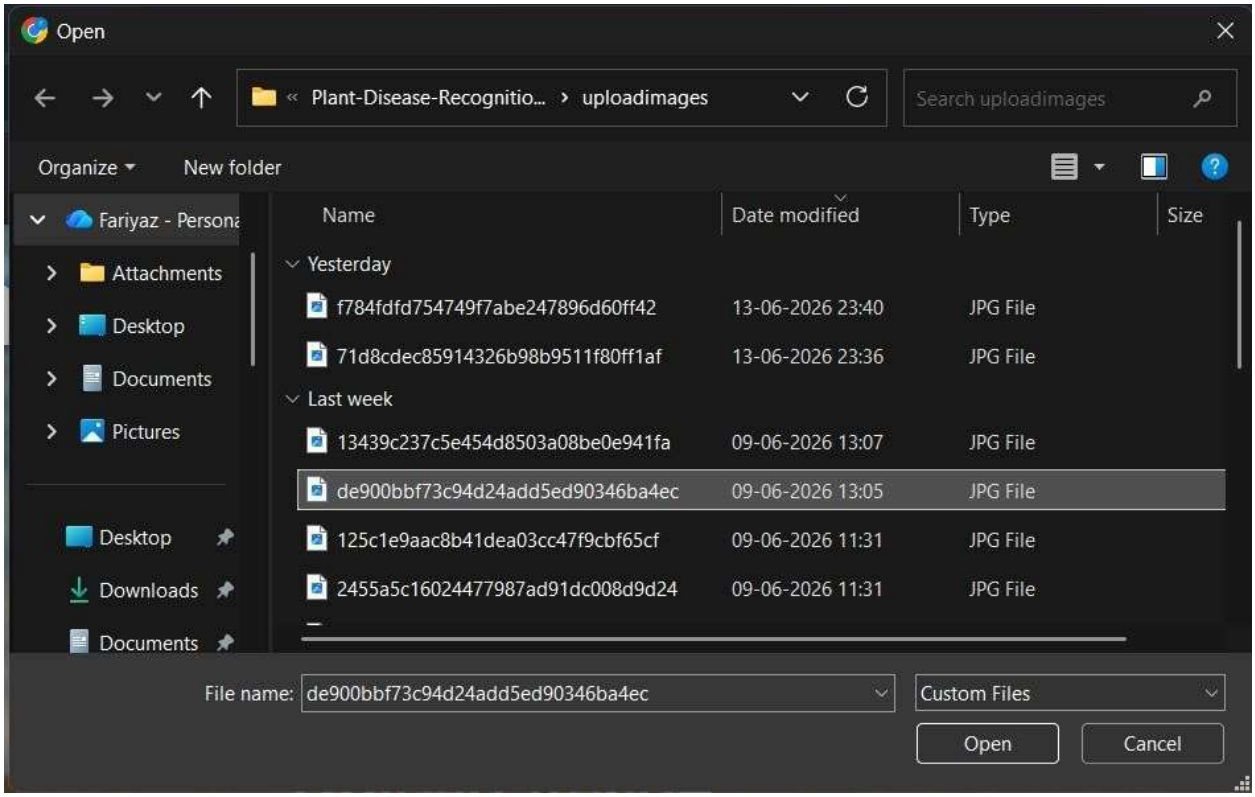

Figure 5.1.2 File Upload

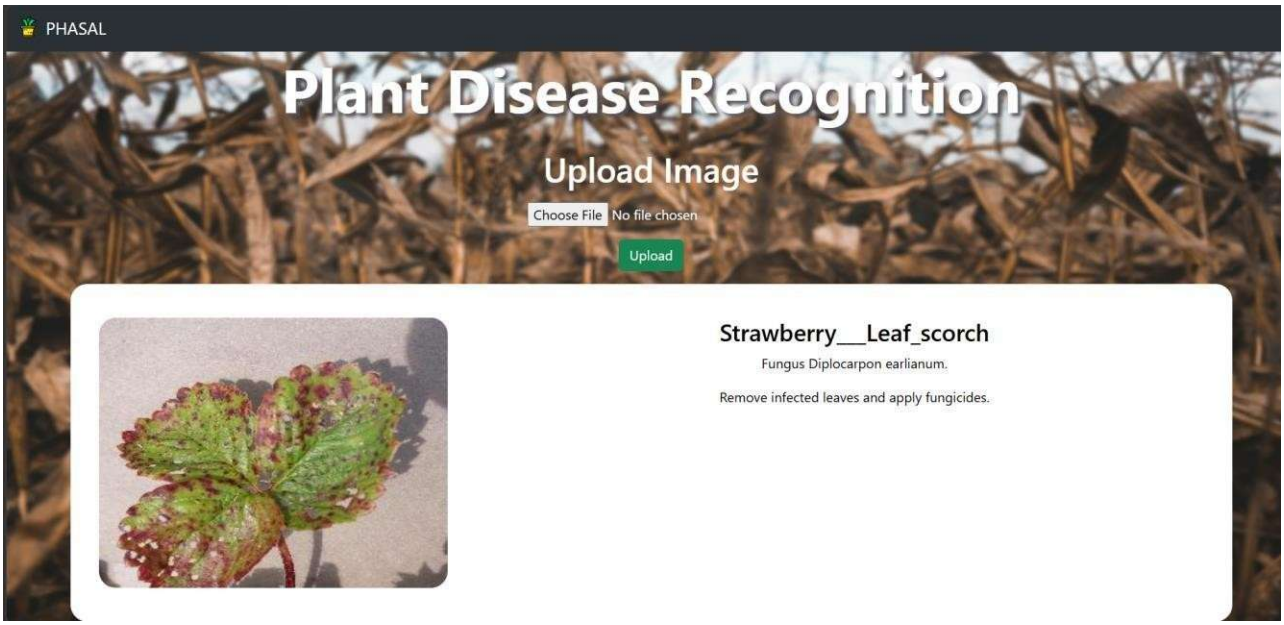

Figure 5.1.3 Result Output

This figure shows the disease prediction result generated by the Plant Disease Recognition System after uploading an infected plant leaf image. The CNN model analyzes the uploaded image and identifies the disease category. The system displays the predicted disease name along with its cause and recommended treatment measures. This demonstrates the ability of the model to accurately detect plant diseases and provide useful guidance to users.

Test Image 2)

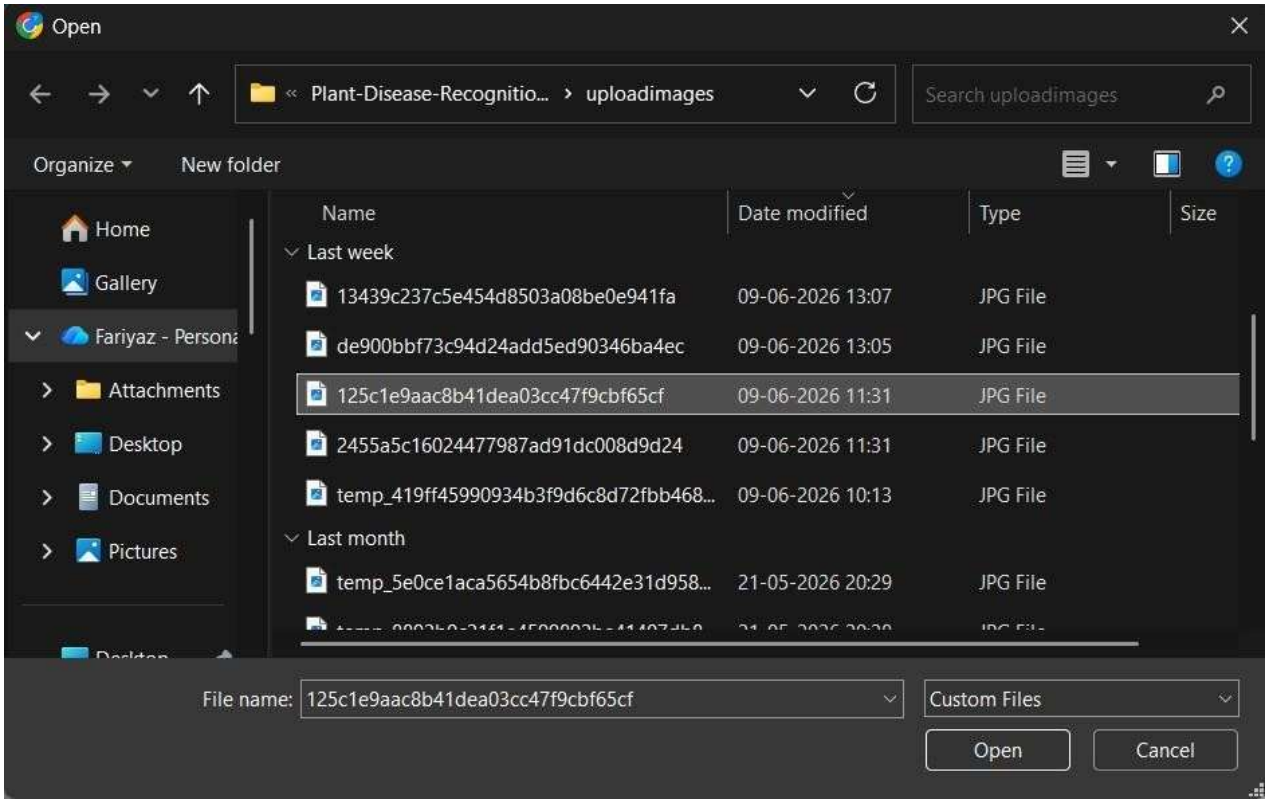

Figure 5.1.4 File Upload

Figure 5.1.5 Result Output

This figure shows the prediction result generated by the Plant Disease Recognition System after uploading a plant leaf image affected by Corn Common Rust. The CNN model successfully identifies the disease as Corn Common Rust, which is caused by the fungus Puccinia sorghi. The system also provides the recommended treatment, including the application of fungicides and the use of resistant plant varieties. This result demonstrates the effectiveness of the deep learning model in accurately detecting corn leaf diseases from uploaded images.

5.2 CODE

Data Preprocessing

```
train_dir = "/content/dataset/train"
validation_dir = "/content/dataset/val"

BATCH_SIZE = 32
IMG_SIZE = (160, 160)

train_dataset = tf.keras.utils.image_dataset_from_directory(train_dir,
                                                            shuffle=True,
                                                            batch_size=BATCH_SIZE,
                                                            image_size=IMG_SIZE)
```

```
Found 49179 files belonging to 39 classes.
```

```
validation_dataset = tf.keras.utils.image_dataset_from_directory(validation_dir,
                                                                 shuffle=True,
                                                                 batch_size=BATCH_SIZE,
                                                                 image_size=IMG_SIZE)
test_dir = "/content/dataset/test"
test_dataset = tf.keras.utils.image_dataset_from_directory(test_dir,
                                                           batch_size=BATCH_SIZE,
                                                           image_size=IMG_SIZE)
```

```
Found 6139 files belonging to 39 classes.
Found 6168 files belonging to 39 classes.
```

Figure 5.2.1 Data split

```
model.compile(optimizer=tf.keras.optimizers.Adam(),
              loss=tf.keras.losses.SparseCategoricalCrossentropy(),
              metrics=[tf.keras.metrics.SparseCategoricalAccuracy(name='accuracy')])
```

```
initial_epochs = 6

loss0, accuracy0 = model.evaluate(validation_dataset)
```

```
192/192 ━━━━━━━━━━━━━━━━━━━━ 50s 138ms/step - accuracy: 0.0215 - loss: 3.8072
```

Figure 5.2.2 Validation Accuracy

Create the base model from the pre-trained convnets

```
IMG_SHAPE = IMG_SIZE + (3,)
base_model = tf.keras.applications.EfficientNetB4(
    input_shape=IMG_SHAPE,
    include_top=False,
    weights='imagenet',
)
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb4_notop.h5
71686520/71686520 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

```
image_batch, label_batch = next(iter(train_dataset))
feature_batch = base_model(image_batch)
print(feature_batch.shape)
```

```
(32, 5, 5, 1792)
```

Figure 5.2.3: Initialization of the Base Model

```
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0
efficientnetb4 (Functional)	(None, 5, 5, 1792)	17,673,823
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1792)	0
dropout (Dropout)	(None, 1792)	0
dense (Dense)	(None, 39)	69,927

Total params: 17,743,750 (67.69 MB)
Trainable params: 69,927 (273.15 KB)
Non-trainable params: 17,673,823 (67.42 MB)

Figure 5.2.4: Model summary

```
print("initial loss: {:.2f}".format(loss0))
print("initial accuracy: {:.2f}".format(accuracy0))
```

```
initial loss: 3.81
initial accuracy: 0.02
```

```
history = model.fit(train_dataset,
                    epochs=initial_epochs,
                    validation_data=validation_dataset)
```

```
Epoch 1/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 164s 77ms/step - accuracy: 0.8198 - loss: 0.7154 - val_accuracy: 0.9107 - val_loss: 0.3241
Epoch 2/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 93s 60ms/step - accuracy: 0.9095 - loss: 0.3160 - val_accuracy: 0.9376 - val_loss: 0.2247
Epoch 3/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 93s 61ms/step - accuracy: 0.9250 - loss: 0.2512 - val_accuracy: 0.9436 - val_loss: 0.1884
Epoch 4/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 95s 62ms/step - accuracy: 0.9332 - loss: 0.2199 - val_accuracy: 0.9449 - val_loss: 0.1668
Epoch 5/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 140s 60ms/step - accuracy: 0.9385 - loss: 0.1995 - val_accuracy: 0.9511 - val_loss: 0.1546
Epoch 6/6
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 94s 61ms/step - accuracy: 0.9413 - loss: 0.1835 - val_accuracy: 0.9559 - val_loss: 0.1400
```

Figure 5.2.5: Initial Training Phase

```
fine_tune_epochs = 10
total_epochs =  initial_epochs + fine_tune_epochs

history_fine = model.fit(train_dataset,
                         epochs=total_epochs,
                         initial_epoch=len(history.epoch),
                         validation_data=validation_dataset)
```

```
Epoch 7/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 454s 214ms/step - accuracy: 0.9353 - loss: 0.2043 - val_accuracy: 0.9438 - val_loss: 0.1037
Epoch 8/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 290s 158ms/step - accuracy: 0.9672 - loss: 0.0888 - val_accuracy: 0.9050 - val_loss: 0.8393
Epoch 9/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9702 - loss: 0.0702 - val_accuracy: 0.9676 - val_loss: 0.0830
Epoch 10/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9694 - loss: 0.0594 - val_accuracy: 0.9730 - val_loss: 0.0553
Epoch 11/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9737 - loss: 0.0485 - val_accuracy: 0.9480 - val_loss: 0.0936
Epoch 12/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9752 - loss: 0.0447 - val_accuracy: 0.9216 - val_loss: 0.0938
Epoch 13/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 247s 161ms/step - accuracy: 0.9752 - loss: 0.0343 - val_accuracy: 0.9578 - val_loss: 0.0668
Epoch 14/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9749 - loss: 0.0334 - val_accuracy: 0.9407 - val_loss: 0.1341
Epoch 15/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 161ms/step - accuracy: 0.9770 - loss: 0.0258 - val_accuracy: 0.9573 - val_loss: 0.0625
Epoch 16/16
1537/1537 ━━━━━━━━━━━━━━━━━━━━ 248s 162ms/step - accuracy: 0.9697 - loss: 0.0291 - val_accuracy: 0.9705 - val_loss: 0.0458
```

Figure 5.2.6: Preparing the Model for Fine-Tuning

```
# Let's take a look to see how many layers are in the base model
print("Number of layers in the base model: ", len(base_model.layers))

# Fine-tune from this layer onwards
fine_tune_at = 100

# Freeze all the layers before the `fine_tune_at` layer
for layer in base_model.layers[:fine_tune_at]:
  layer.trainable = False
```

```
Number of layers in the base model:  475
```

Figure 5.2.7: Fine-Tuning the Model

```
# Retrieve a batch of images from the test set
image_batch, label_batch = test_dataset.as_numpy_iterator().next()
predictions = model.predict_on_batch(image_batch)
predictions = tf.argmax(predictions,axis=1)

print('Predictions:\n', predictions.numpy())
print('Labels:\n', label_batch)

plt.figure(figsize=(10, 10))
for i in range(9):
  ax = plt.subplot(3, 3, i + 1)
  plt.imshow(image_batch[i].astype("uint8"))
  plt.title(class_names[predictions[i]])
  plt.axis("off")
```

```
Predictions:
 [ 9 21 38 34 25  2 23 29 35  0  9  1 38 26 16 20 20  5 36 36  6 37 36 26
 31 16 36 26  5 16 23 36]
Labels:
 [ 9 21 38 34 25  2 23 29 35  0  9  1 38 26 16 20 20  5 36 36  6 37 36 26
 31 16 36 26  5 16 23 36]
```

Figure 5.2.8: Model Evaluation and Prediction

```
global_average_layer = tf.keras.layers.GlobalAveragePooling2D()
feature_batch_average = global_average_layer(feature_batch)
print(feature_batch_average.shape)
```

```
(32, 1792)
```

```
prediction_layer = tf.keras.layers.Dense(len(class_names), activation='sigmoid')
prediction_batch = prediction_layer(feature_batch_average)
print(prediction_batch.shape)
```

```
(32, 39)
```

Figure 5.2.9: Constructing the Classification Head

```
loss, accuracy = model.evaluate(test_dataset)
print('Test accuracy :', accuracy)
```

```
193/193 ━━━━━━━━━━━━━━━━━━━━ 29s 153ms/step - accuracy: 0.9763 - loss: 0.0377
Test accuracy : 0.9763294458389282
```

```
print("initial loss: {:.2f}".format(loss0))
print("initial accuracy: {:.2f}".format(accuracy0))
```

```
initial loss: 3.81
initial accuracy: 0.02
```

Figure 5.2.10: Accuracy and loss

```
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0
efficientnetb4 (Functional)	(None, 5, 5, 1792)	17,673,823
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1792)	0
dropout (Dropout)	(None, 1792)	0
dense (Dense)	(None, 39)	69,927

Total params: 17,743,750 (67.69 MB)
Trainable params: 17,531,783 (66.88 MB)
Non-trainable params: 211,967 (828.00 KB)

Figure 5.2.10: Mode; Summary

```
class_names = train_dataset.class_names

plt.figure(figsize=(10, 10))
for images, labels in train_dataset.take(1):
  for i in range(9):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(images[i].numpy().astype("uint8"))
    plt.title(class_names[labels[i]])
    plt.axis("off")
```

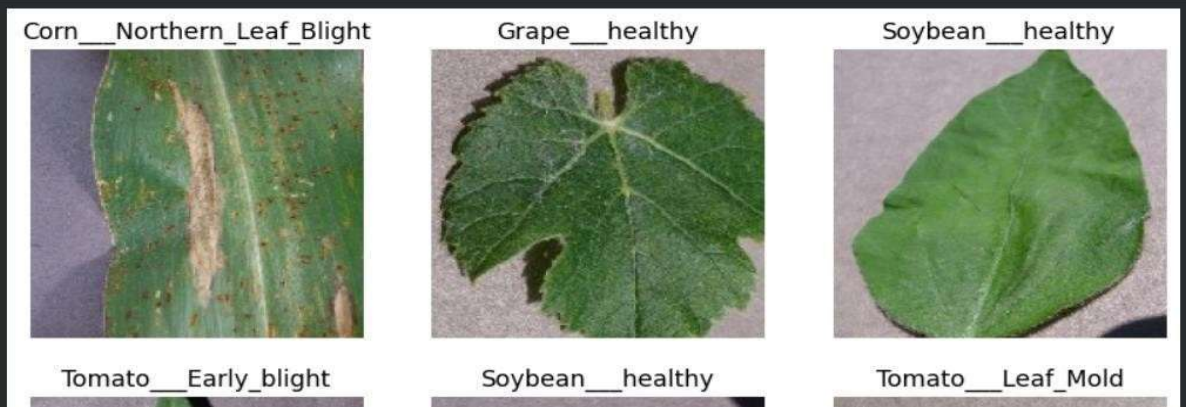

Figure 5.2.11: Training Dataset Visualization

```
{0: 'Apple___Apple_scab',
 1: 'Apple___Black_rot',
 2: 'Apple___Cedar_apple_rust',
 3: 'Apple___healthy',
 4: 'Blueberry___healthy',
 5: 'Cherry_(including_sour)___Powdery_mildew',
 6: 'Cherry_(including_sour)___healthy',
 7: 'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot',
 8: 'Corn_(maize)___Common_rust_',
 9: 'Corn_(maize)___Northern_Leaf_Blight',
 10: 'Corn_(maize)___healthy',
 11: 'Grape___Black_rot',
 12: 'Grape___Esca_(Black_Measles)',
 13: 'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)',
 14: 'Grape___healthy',
 15: 'Orange___Haunglongbing_(Citrus_greening)',
 16: 'Peach___Bacterial_spot',
 17: 'Peach___healthy',
 18: 'Pepper,_bell___Bacterial_spot',
 19: 'Pepper,_bell___healthy',
 20: 'Potato___Early_blight',
 21: 'Potato___Late_blight',
 22: 'Potato___healthy',
 23: 'Raspberry___healthy',
 24: 'Soybean___healthy',
...
 33: 'Tomato___Spider_mites Two-spotted_spider_mite',
 34: 'Tomato___Target_Spot',
 35: 'Tomato___Tomato_Yellow_Leaf_Curl_Virus',
 36: 'Tomato___Tomato_mosaic_virus',
 37: 'Tomato___healthy'}
```

Figure 5.2.12: classes

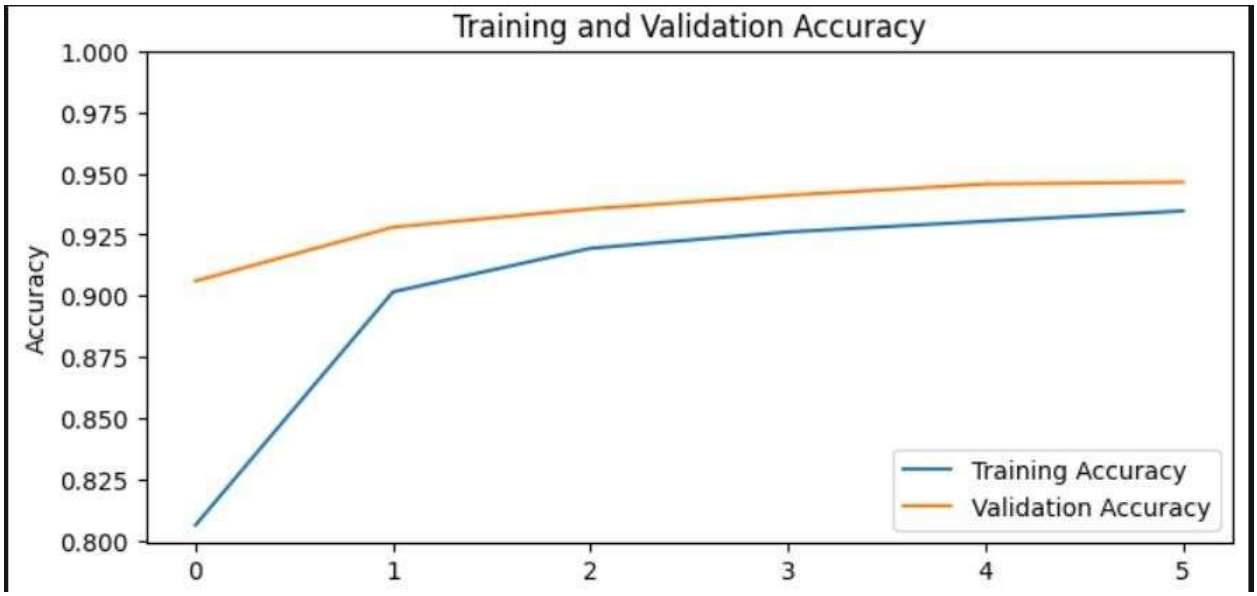

Figure 5.2.13: Model learning performance metrics (i)

Figure 5.2.14: Model learning performance metrics (ii)

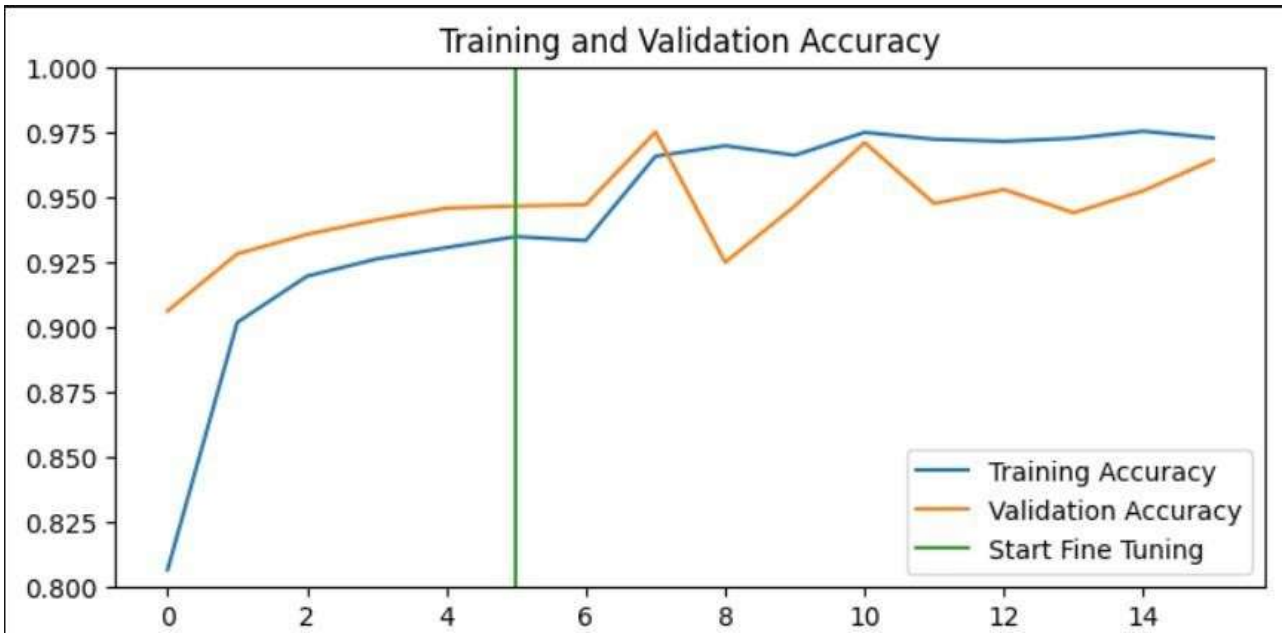

Figure 5.2.15: Model learning performance metrics (iii)

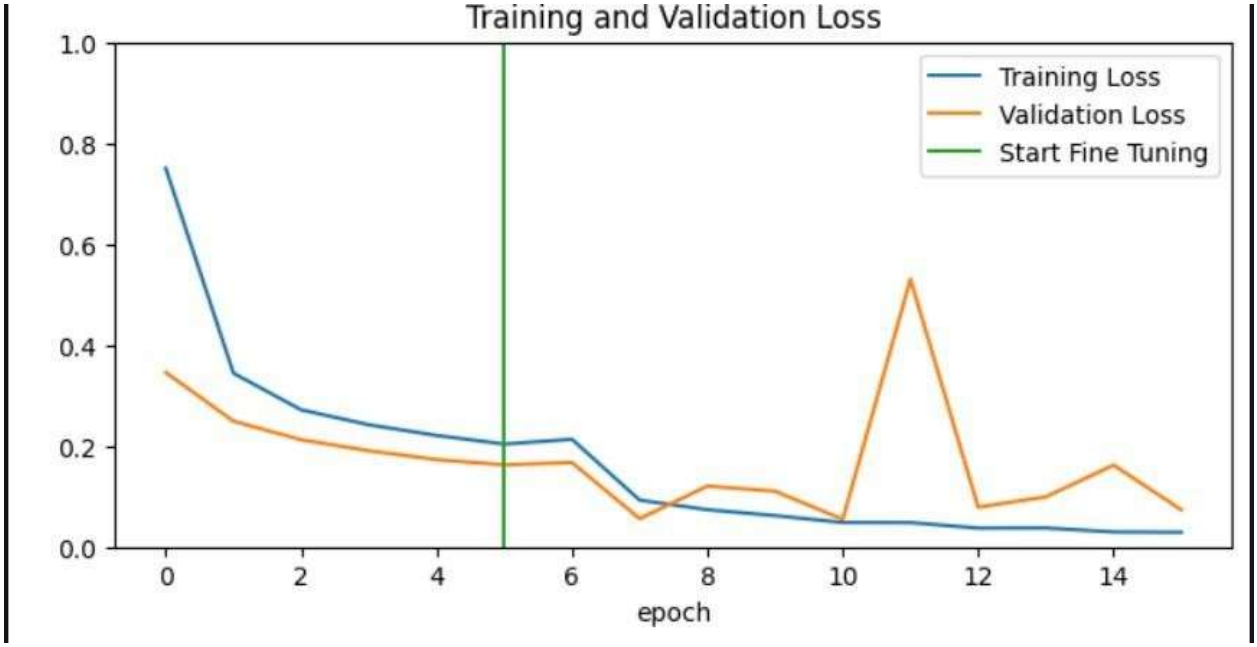

Figure 5.2.16: Model learning performance metrics (iv)

5.3 Code for UI :

app2.py > ...

```python
from flask import Flask, render_template, request, redirect, send_from_directory
from werkzeug.utils import secure_filename
import tensorflow as tf
import numpy as np
import json
import uuid
import os

# ==========================
# FLASK APP
# ==========================
app = Flask(__name__)
```

Figure 5.3.1 Import libraries

```python
# ==========================
# PROJECT PATHS
# ==========================
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploadimages")
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

MODEL_PATH = os.path.join(
    BASE_DIR,
    "models",
    "plant_disease_recog_model_pwp.keras"
)

JSON_PATH = os.path.join(
    BASE_DIR,
    "plant_disease.json"
)
```

Figure 5.3.2 upload path

```
# LOAD MODEL
# ============================
print("Loading model...")

model = tf.keras.models.load_model(
    MODEL_PATH,
    compile=False
)

print("Model loaded successfully!")
print("Input Shape:", model.input_shape)
```

Figure 5.3.3 upload file

```
# LOAD JSON
# ============================
try:
    with open(JSON_PATH, "r", encoding="utf-8") as f:
        plant_disease = json.load(f)

    print("JSON loaded successfully")

except Exception as e:
    print("JSON Error:", e)
    plant_disease = {}

# ============================
# ROUTES
# ============================
@app.route('/')
def home():
    return render_template("home.html")


@app.route('/uploadimages/<path:filename>')
def uploaded_images(filename):
    return send_from_directory(
        UPLOAD_FOLDER,
        filename
    )
```

Figure 5.3.4 Connect JSON

```python
# IMAGE PREPROCESSING
# ==========================
def extract_features(image_path):

    try:
        input_shape = model.input_shape

        if len(input_shape) == 4:
            img_height = input_shape[1]
            img_width = input_shape[2]
        else:
            img_height = 224
            img_width = 224

        image = tf.keras.utils.load_img(
            image_path,
            target_size=(img_height, img_width)
        )

        image = tf.keras.utils.img_to_array(image)

        image = image / 255.0

        image = np.expand_dims(
            image,
            axis=0
        )

        return image

    except Exception as e:
        raise Exception(
            f"Image Processing Error: {str(e)}"
        )
```

Figure 5.3.5 Image processing

```python
# ============================
# RUN APP
# ============================
if __name__ == "__main__":

    app.run(
        host="0.0.0.0",
        port=5000,
        debug=True
    )
```

Figure 5.3.6 : Run Application

CHAPTER 6

CONCLUSION AND FUTURE WORK

CONCLUSION:

The Plant Disease Recognition System was successfully developed using Deep Learning techniques and a Convolutional Neural Network (CNN) model. The system is capable of automatically detecting and classifying plant diseases from leaf images with high accuracy. The trained model was integrated with a Flask-based web application, allowing users to upload leaf images and obtain disease predictions in a simple and user-friendly manner.

The proposed system performs image preprocessing, disease classification, and retrieval of disease information and remedies from a JSON database. Experimental results showed that the model can effectively identify different plant diseases and provide appropriate treatment recommendations. The achieved accuracy of approximately **97%** demonstrates the effectiveness of CNN in plant disease detection tasks.

FUTURE WORK:

The developed system can assist farmers, agricultural researchers, and students in the early diagnosis of plant diseases, helping to reduce crop losses and improve agricultural productivity. Overall, the project provides an efficient, reliable, and cost-effective solution for automated plant disease recognition and management.

Although the proposed Plant Disease Recognition System achieves high accuracy in disease detection, there are several areas where the system can be further improved and expanded in the future.

1. The system can be enhanced by increasing the size and diversity of the training dataset to improve prediction accuracy for a larger number of plant species and diseases.
2. Real-time disease detection can be implemented using mobile phone cameras, allowing farmers to identify diseases directly in the field.
3. The application can be converted into an Android or iOS mobile application for easier accessibility and wider adoption.
4. Additional information such as pesticide recommendations, fertilizer suggestions, and preventive measures can be incorporated into the system.

5. Advanced Deep Learning architectures such as ResNet, EfficientNet, or Vision Transformers can be used to further improve classification performance.
6. The system can be integrated with cloud-based services to provide centralized data storage and remote access to disease prediction results.
7. Multi-language support can be added to help farmers from different regions understand disease information and treatment recommendations more effectively.
8. Weather and environmental data can be integrated into the system to provide early disease warnings and preventive recommendations.
9. The system can be extended to detect diseases in fruits, stems, flowers, and other plant parts in addition to leaves.
10. Integration with IoT sensors and smart agriculture technologies can help create a complete intelligent crop monitoring and disease management system.
11. Thus, the proposed system provides a strong foundation for automated plant disease diagnosis and offers significant opportunities for future enhancements in precision agriculture.

REFERENCES

1. Mrunalini R. et al., An application of K-means clustering and artificial intelligence in pattern recognition for crop diseases ,2011.

2. S.Raj Kumar , S.Sowrirajan," Automatic Leaf Disease Detection and Classification using Hybrid Features and Supervised Classifier", International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering, vol. 5, Issue 6,2016.

3. Tatem, D. J. Rogers, and S. I. Hay, "Global transport networks and infectious disease spread," Advances in Parasitology, vol. 62, pp. 293–343, 2006. View at Publisher · View at Google Scholar · View at Scopus.

4. J. R. Rohr, T. R. Raffel, J. M. Romansic, H. McCallum, and P. J. Hudson, "Evaluating the links between climate, disease spread, and amphibian declines," Proceedings of the National Academy of Sciences of the United States of America, vol. 105, no. 45, pp. 17436–17441, 2008. View at Publisher · View at Google Scholar View at Scopus.

5. Robert G. de Luna, Elmer P. Dadios, and Argel A. Bandala, "Automated Image Capturing System for Deep Learning-Based Tomato Plant Leaf Disease Detection and Recognition," International Conference on Advances in Big Data, Computing and Data Communication Systems, 2019.

6. H. Cartwright, Ed., Artificial Neural Networks, Humana Press, 2015.

7. Steinwart and A. Christmann, Support Vector Machines, Springer Science & Business Media, New York, NY, USA, 2008. View at MathSciNet.

8. Steinwart and A. Christmann, Support Vector Machines, Springer Science & Business Media, New York, NY, USA, 2008. View at MathSciNet.

9. S. Sankaran, A. Mishra, R. Ehsani, and C. Davis, "A review of advanced techniques for detecting plant diseases," Computers and Electronics in Agriculture, vol. 72, no. 1, pp. 1–13, 2010. View at Publisher ·View at Google Scholar · View at Scopus.

10. P. R. Reddy, S. N. Divya, and R. Vijayalakshmi, “Plant disease detection technique tool—a theoretical approach,” International Journal of Innovative Technology and Research, pp. 91–93, 2015. View at Google Scholar.

11. Sachin D. Khirade and A. B. Patil, "Plant Disease Detection Using Image Processing," 2015.

12. Amandeep Singh, "Plant Leaf Disease Detection Using Image Processing Techniques," 2016.

13. Satish Madhgoria et al., "Automatic Pixel-Based Detection of Unhealthy Regions in Plant Leaf Images Using SVM Classification," 2017.

14. Omkar Kulkarni, "Crop Disease Detection Using Deep Learning," IEEE, 2018.

15. Karunya Rathan, Abirami Devaraj, K. Indira and Sarvepalli Jaahnavi, "Identification of Plant Disease Using Image Processing Technique," ICCSP, IEEE, 2019.